

OBJECT ORIENTED PROGRAMING

OOP

PRACTICALS

NAME: SYEDA FAIZA ASLAM

SECTION: B

ROLL NO: CT-25064

LAB NO: 10

COURSE CODE: CT-260

COURSE TEACHER: Mr. AHMED ZAKI

Exercise

QUESTION: 1

Create a C++ Program to add, subtract, multiply and divide two numbers using classtemplate. Two numbers can be of the same datatype or combination of different data types.

SOURCECODE:

```
#include<iostream>
using namespace std;

template <class T1, class T2>
class Calculator
{
    T1 a;
    T2 b;

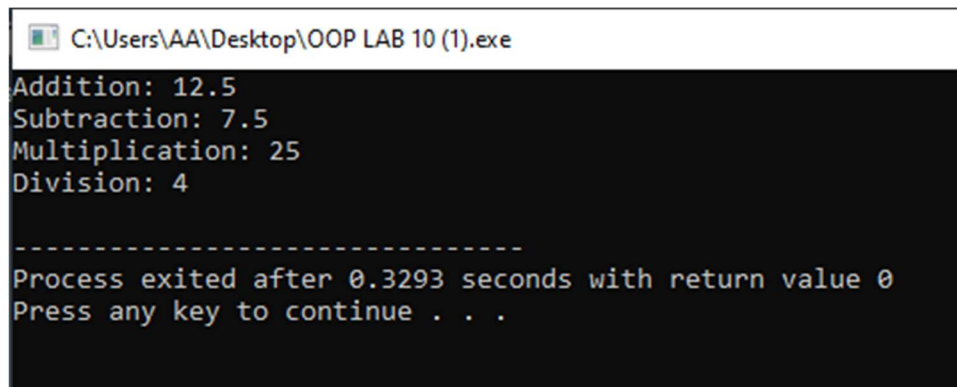
public:
    Calculator(T1 x, T2 y)
    {
        a = x;
        b = y;
    }

    void operations()
    {
        cout << "Addition: " << a + b << endl;
        cout << "Subtraction: " << a - b << endl;
        cout << "Multiplication: " << a * b << endl;
        cout << "Division: " << a / b << endl;
    }
};
```

```
int main()
{
    Calculator<int, float> obj(10, 2.5);
    obj.operations();

    return 0;
}
```

OUTPUT:



```
C:\Users\AA\Desktop\OOP LAB 10 (1).exe
Addition: 12.5
Subtraction: 7.5
Multiplication: 25
Division: 4

-----
Process exited after 0.3293 seconds with return value 0
Press any key to continue . . .
```

QUESTION: 2

Create a C++ Program to swap the data using template function, instead of calling a function by passing a value, use call by reference, two numbers can be of same datatype or combination of different data types.

SOURCECODE:

```
#include<iostream>
using namespace std;

template <class T1, class T2>
void swapValues(T1 &a, T2 &b)
{
    auto temp = a;
    a = b;
    b = temp;
}

int main()
{
    int x = 5;
    float y = 10.5;

    cout << "Before Swap: " << x << " " << y << endl;

    swapValues(x, y);

    cout << "After Swap: " << x << " " << y << endl;

    return 0;
}
```

OUTPUT:

```
C:\Users\AA\Desktop\OOP LAB 10 (2).exe
Before Swap: 5 10.5
After Swap: 10 5

-----
Process exited after 0.4021 seconds with return value 0
Press any key to continue . . .
```

QUESTION: 3

Create a C++ class called mycontainer that can store one element of any type and it has just one member function called increase, which increases its value. But we find that when it stores an element of type char it would be more convenient to have a completely different implementation with a function member uppercase, declare a class template specialization for that type.

SOURCECODE:

```
#include<iostream>
using namespace std;
```

```
template <class T>
class mycontainer
{
    T value;
```

```
public:
```

```
    mycontainer(T x)
    {
        value = x;
    }
```

```
    T increase()
```

```
    {  
        return ++value;  
    }  
};
```

```
// Specialization for char  
template <>
```

```
class mycontainer<char>  
{  
    char value;
```

```
public:
```

```
    mycontainer(char x)  
    {  
        value = x;  
    }
```

```
    char uppercase()  
    {  
        if(value >= 'a' && value <= 'z')  
            value = value - 32;  
  
        return value;  
    }
```

```
};
```

```
int main()
```

```
{  
    mycontainer<int> obj1(5);  
    cout << "Increased: " << obj1.increase() << endl;  
  
    mycontainer<char> obj2('a');  
    cout << "Uppercase: " << obj2.uppercase() << endl;  
  
    return 0;
```

```
}
```

OUTPUT:

```
C:\Users\AA\Desktop\OOP LAB 10 (3).exe
Increased: 6
Uppercase: A

-----
Process exited after 0.3274 seconds with return value 0
Press any key to continue . . .
```

QUESTION: 4

Create an abstract class template for 1D dynamic array. Inherit a dynamic Queue template class from the abstract class which must have following methods: isFull(), isEmpty(), size(), Front(), Rear(), enqueue(), dequeue(), resize().

SOURCECODE:

```
#include<iostream>
using namespace std;

template <class T>
class Array
{
public:
    virtual bool isFull() = 0;
    virtual bool isEmpty() = 0;
    virtual int size() = 0;
};

template <class T>
class Queue : public Array<T>
```

```

{
    T *arr;
    int front, rear, capacity;

public:
    Queue(int size = 5)
    {
        capacity = size;
        arr = new T[capacity];
        front = -1;
        rear = -1;
    }

    bool isFull()
    {
        return (rear == capacity - 1);
    }

    bool isEmpty()
    {
        return (front == -1 || front > rear);
    }

    int size()
    {
        if(isEmpty()) return 0;
        return rear - front + 1;
    }

    T Front()
    {
        return arr[front];
    }

    T Rear()

```



```

{
    return arr[rear];
}

void enqueue(T value)
{
    if(isFull())
    {
        resize();
    }

    if(front == -1) front = 0;
    arr[++rear] = value;
}

void dequeue()
{
    if(isEmpty())
    {
        cout << "Queue Empty\n";
        return;
    }
    front++;
}

void resize()
{
    int newCap = capacity * 2;
    T *newArr = new T[newCap];

    for(int i = 0; i < capacity; i++)
        newArr[i] = arr[i];

    delete[] arr;
    arr = newArr;
}

```

```

        capacity = newCap;
    }
};

int main()
{
    Queue<int> q;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    cout << "Front: " << q.Front() << endl;
    cout << "Rear: " << q.Rear() << endl;
    cout << "Size: " << q.size() << endl;

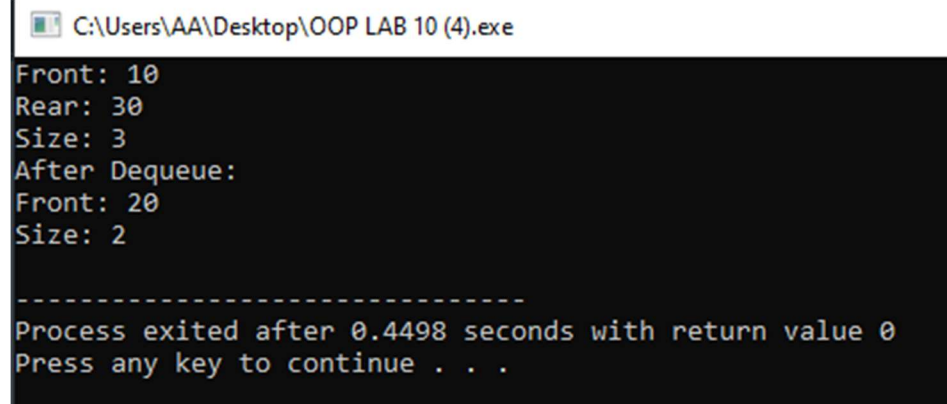
    q.dequeue();

    cout << "After Dequeue:\n";
    cout << "Front: " << q.Front() << endl;
    cout << "Size: " << q.size() << endl;

    return 0;
}

```

OUTPUT:



```

C:\Users\AA\Desktop\OOP LAB 10 (4).exe
Front: 10
Rear: 30
Size: 3
After Dequeue:
Front: 20
Size: 2

-----
Process exited after 0.4498 seconds with return value 0
Press any key to continue . . .

```

QUESTION: 5

A print shop receives print job requests from various clients throughout the day. The print shop wants to efficiently manage the print jobs and ensure they are printed in the correct order.

- Create an object of class queue that you have just created to represent the printer job queue.
- Whenever a print job request arrives, enqueue (add) the job to the back of the queue.
- The print shop has a printer that can handle one job at a time. If the printer is idle and there are jobs in the queue, dequeue (remove) the job from the front of the queue and start printing it.
- Once a print job is completed, check if there are more jobs in the queue. If there are, dequeue the next job and start printing it. If the queue is empty, the printer remains idle.
- Repeat steps 3 and 4 until all print jobs have been completed.

This scenario can be solved using a queue because it follows the First-In-First-Out (FIFO) principle. The print jobs are handled in the same order they arrived at the print shop. The queue ensures that jobs are printed in the correct sequence, without skipping or rearranging the order in which they were received.

SOURCECODE:

```
#include<iostream>
using namespace std;
```

```
template <class T>
class Queue
{
    T arr[100];
    int front, rear;
```

```

public:
    Queue()
    {
        front = -1;
        rear = -1;
    }

    bool isEmpty()
    {
        return (front == -1 || front > rear);
    }

    void enqueue(T value)
    {
        if(front == -1) front = 0;
        arr[++rear] = value;
    }

    T dequeue()
    {
        return arr[front++];
    }
};

int main()
{
    Queue<string> printerQueue;

    // Adding jobs
    printerQueue.enqueue("Job1");
    printerQueue.enqueue("Job2");
    printerQueue.enqueue("Job3");

    cout << "Printing Jobs...\n";
}

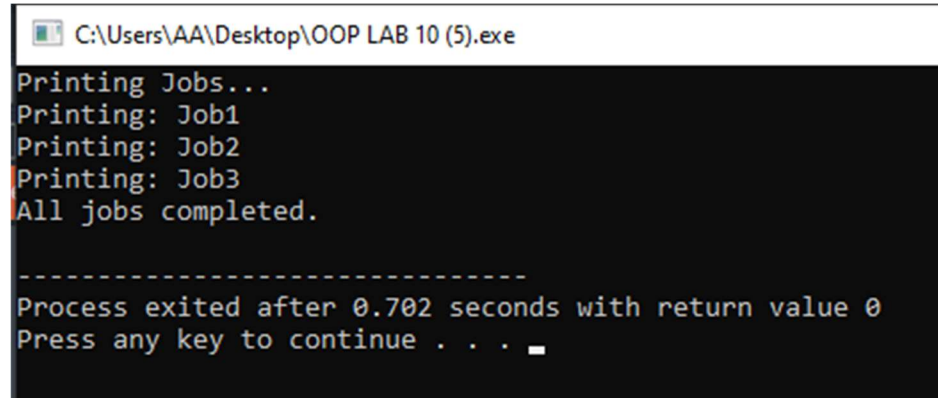
```

```
while(!printerQueue.isEmpty())
{
    string job = printerQueue.dequeue();
    cout << "Printing: " << job << endl;
}

cout << "All jobs completed.\n";

return 0;
}
```

OUTPUT:



```
C:\Users\AA\Desktop\OOP LAB 10 (5).exe
Printing Jobs...
Printing: Job1
Printing: Job2
Printing: Job3
All jobs completed.

-----
Process exited after 0.702 seconds with return value 0
Press any key to continue . . .
```

SOLVED EXAMPLES

EXAMPLE NO: 1

Template

Function(Single Type)

SOURCECODE:

```
#include<iostream>
using namespace std;
```

```
template <class T>
T Max(T a, T b)
{
    if(a > b)
        return a;
    else
        return b;
}
```

```
int main()
{
    cout << "Max
integer: " << Max(22,
2) << endl;
    cout << "Max float: "
<< Max(3.9, 22.8) <<
endl;

    return 0;
}
```

OUTPUT:

```
(globals)
C:\Users\AA\Desktop\examples.exe
Max integer: 22
Max float: 22.8

-----
Process exited after 0.3216 seconds with return value 0
Press any key to continue . . .
```

EXAMPLE NO: 2

Template Function (Multiple Types)

SOURCECODE:

```
#include<iostream>
using namespace std;
```

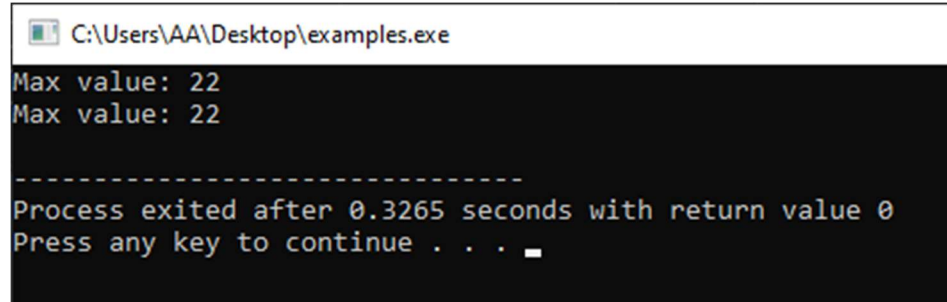
```
template <class T1,
class T2>
T1 Max(T1 a, T2 b)
{
    if(a > b)
        return a;
    else
        return b;
}
```

```
int main()
{
    cout << "Max value:
" << Max(22, 2.77) <<
endl;
```

```
    cout << "Max value:
" << Max(3.9, 22) <<
endl;

    return 0;
}
```

OUTPUT:



```
C:\Users\AA\Desktop\examples.exe
Max value: 22
Max value: 22

-----
Process exited after 0.3265 seconds with return value 0
Press any key to continue . . .
```

EXAMPLE NO: 3

Class Template **(mypair)**

SOURCECODE:

```
#include<iostream>
using namespace std;

template <class T>
class mypair
{
    T a, b;

public:
    mypair(T first, T
second)
    {
        a = first;
        b = second;
```



```

    }

    T getMax()
    {
        if(a > b)
            return a;
        else
            return b;
    }
};

int main()
{
    mypair<int> obj(100,
75);
    cout << "Max value:
" << obj.getMax();

    return 0;
}

```

OUTPUT:

 C:\Users\AA\Desktop\examples.exe

```

Max value: 100
-----
Process exited after 0.4008 seconds with return value 0
Press any key to continue . . .

```

EXAMPLE NO: 4

Multiple Argument

Class Template

SOURCECODE:

```
#include<iostream>
using namespace std;
```

```
template <class T1,
class T2>
class sample
{
    T1 a;
    T2 b;

public:
    void getdata()
    {
        cout << "Enter
value of a and b: ";
        cin >> a >> b;
    }

    void display()
    {
        cout << "Value of
a: " << a << endl;
        cout << "Value of
b: " << b << endl;
    }
};
```

```
int main()
{
    sample<int, int> s1;
```

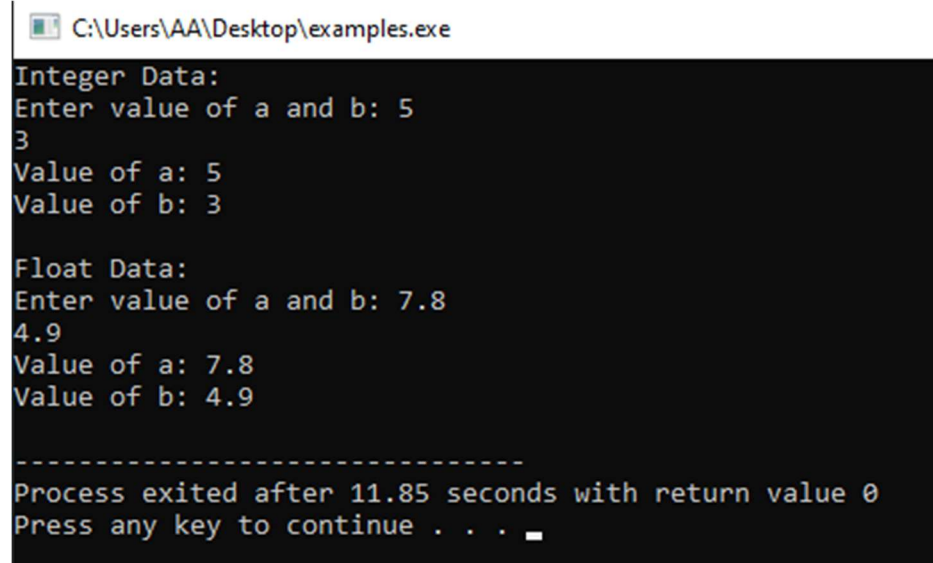
```
    sample<float, float>  
s2;
```

```
    cout << "Integer  
Data:\n";  
    s1.getdata();  
    s1.display();
```

```
    cout << "\nFloat  
Data:\n";  
    s2.getdata();  
    s2.display();
```

```
    return 0;  
}
```

OUTPUT:



```
C:\Users\AA\Desktop\examples.exe  
Integer Data:  
Enter value of a and b: 5  
3  
Value of a: 5  
Value of b: 3  
  
Float Data:  
Enter value of a and b: 7.8  
4.9  
Value of a: 7.8  
Value of b: 4.9  
  
-----  
Process exited after 11.85 seconds with return value 0  
Press any key to continue . . .
```

EXAMPLE NO: 5

**Template
Specialization
(Function)**

SOURCECODE:

```
#include<iostream>
#include<typeinfo>
using namespace std;
```

```
template <class T>
void printFunction(T
arg)
{
    cout << "Generic
Function: " << arg <<
endl;
}
```

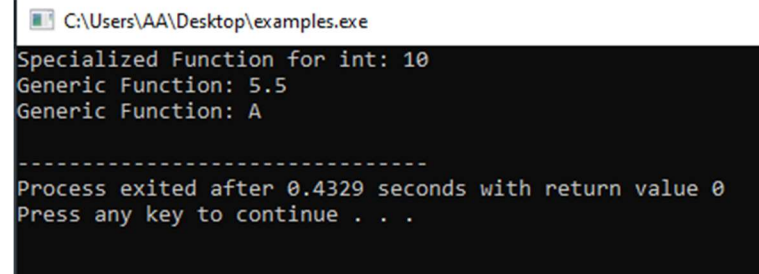
```
// Specialization for int
template <>
void printFunction(int
arg)
{
    cout << "Specialized
Function for int: " <<
arg << endl;
}
```

```
int main()
{
    printFunction(10);
    // int
    printFunction(5.5);
    // double
```

```
    printFunction('A');  
    // char
```

```
    return 0;  
}
```

OUTPUT:



The screenshot shows a Windows command prompt window with the title bar "C:\Users\AA\Desktop\examples.exe". The output text is as follows:

```
Specialized Function for int: 10  
Generic Function: 5.5  
Generic Function: A  
  
-----  
Process exited after 0.4329 seconds with return value 0  
Press any key to continue . . .
```

EXAMPLE NO: 6

Class Template

Specialization (char)

SOURCECODE:

```
#include<iostream>
using namespace std;
```

```
template <class T>
class Test
{
    T value;

public:
    Test(T v)
    {
        value = v;
    }

    void display()
    {
        cout << "Value: "
<< value << endl;
    }
};
```

```
// Specialization for
char
template <>
class Test<char>
{
    char value;

public:
    Test(char v)
```

```

    {
        value = v;
    }

    void display()
    {
        if(value >= 'a' &&
value <= 'z')
            value = value -
32;

        cout <<
"Uppercase: " << value
<< endl;
    }
};

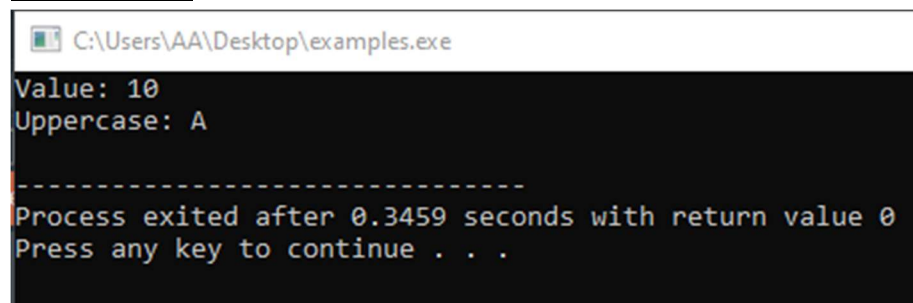
int main()
{
    Test<int> obj1(10);
    obj1.display();

    Test<char> obj2('a');
    obj2.display();

    return 0;
}

```

OUTPUT:



```

C:\Users\AA\Desktop\examples.exe
Value: 10
Uppercase: A

-----
Process exited after 0.3459 seconds with return value 0
Press any key to continue . . .

```